

Pacote 3 — Governança, Permissões e TI

Documentação **autocontida** para super administrador, TI e desenvolvedor.

Atualizado em: 22/05/2026

Conteúdo integrado: Manual operacional ACL · Modelo de controle de acesso · Blueprint completo

Parte 1 — Manual operacional de Controle de Acesso

Manual operacional — Controle de acesso (app-igreja)

Este manual descreve **como operar** o controle de acesso no dia a dia: instalação, atribuição de papéis, ajuste de permissões, testes e resolução de problemas.

Documentação técnica de referência: [CONTROLE_ACESSO.md](#)

Pacote: [PACOTE_3_GOVERNANCA_TI.md](#) · **Índice:** [INDICE_DOCUMENTACAO.md](#)

Atualizado em: 22/05/2026

1. Para quem é este manual

Público	Uso
Super administrador	Configura papéis e permissões pela UI ou SQL
Equipe de TI / secretaria	Atribui events_admin, pastoral, etc. a pessoas certas
Desenvolvedor	Instala scripts, valida RLS e RPCs após deploy

2. Conceitos essenciais

2.1 Identidade

- O “usuário” do ACL é sempre **profiles.id** (UUID).
 - Login no app: telefone + PIN de 4 dígitos (**profiles.access_pin**).
-

- Após o login, o app grava `user_profile_id` na sessão e envia o header `x-profile-id` ao Supabase.

2.2 Papéis (`access_roles`)

Um perfil pode ter **vários papéis** ao mesmo tempo (ex.: `member` + `events_admin`).

Código	Nome	Uso típico
<code>visitantes</code>	Visitantes	Acesso público mínimo; fallback sem perfil na sessão ou perfil sem papéis
<code>congregado</code>	Congregado	Participante com acesso básico (sem gerenciar família)
<code>member</code>	Membro	Acesso padrão do aplicativo
<code>family_acceptor</code>	Responsável familiar	Gerencia família (complementar ao <code>member</code>)
<code>lider</code>	Líder	Gerencia servos e programação dos tipos de escala atribuídos ao perfil
<code>events_admin</code>	Administrador de eventos	Manutenção de eventos e salas
<code>pastoral</code>	Equipe pastoral	Triagem de pedidos pastorais
<code>super_admin</code>	Super administrador	Configura o ACL; acesso amplo

Ordem no painel (aba Papéis e lista de papéis do perfil): Visitantes → Congregado → Membro → Responsável familiar → Líder → Administrador de eventos → Equipe pastoral → Super administrador.

2.4 Líder de escala (`lider`)

O papel `lider` permite gerenciar **tipos específicos** de escala (ex.: Vigilância, Recepção):

1. **Papéis** → ajuste grants do `lider` (painéis `maintenance.card.scale_volunteers` e `maintenance.card.scales`).
2. **Perfis** → atribua o papel `lider` ao perfil.
3. **Perfis** → na seção **Liderança por tipo de escala**, ligue os tipos que essa pessoa comanda.

Recursos por tipo: `screen:scale_type.<codigo>` (criados automaticamente a partir de `tipos_escala.codigo`).

Quem tem `maintenance.card.scale_types` (ou `super_admin`) cria/edita tipos; o líder só opera nos tipos vinculados.

Script: `scripts/access-control-lider-escala.sql`

Regra: todo perfil ativo deve manter pelo menos o papel `member` , salvo exceções administrativas explícitas.

Fallback Visitantes: quem **não tem** `user_profile_id` **na sessão** (antes do login) ou tem perfil **sem nenhum papel** em `profile_access_roles` recebe automaticamente os grants do papel `visitantes` — não é necessário atribuir esse papel manualmente na maioria dos casos.

2.3 Recursos (`access_resources`)

O que pode ser protegido:

Tipo	Exemplo de chave	O que controla
screen	<code>/manage-profile</code>	Abrir uma tela ou card do dashboard
table	<code>profiles</code>	Leitura/gravação em uma tabela
column	<code>profiles.cpf</code>	Ver/editar um campo em Dados cadastrais

Curingas (uso restrito):

- `screen:*` — todas as telas
- `table:*` — todas as tabelas
- `column:profiles.*` — todas as colunas de `profiles`

2.4 Permissões (`access_grants`)

Cada grant liga um **papel** (ou um perfil específico) a um **recurso**:

Flag	Significado
<code>can_view</code>	Pode ver (tela, listagem, campo)
<code>can_update</code>	Pode alterar (formulário, UPDATE, RPC de escrita)

Importante: `can_view` em `table:profiles` **não** libera automaticamente `profiles.cpf` ou `profiles.access_pin` . Colunas sensíveis exigem grants de coluna explícitos.

2.5 Onde a permissão é aplicada

```
flowchart TB
    subgraph app [App]
        G[Guard de rota screen]
        C[Campos visíveis em Dados cadastrais]
        K[Cards do dashboard]
    end
    end
```

```

subgraph supabase [Supabase]
  R[RLS nas tabelas]
  P[RPCs update_profile_field / PIN]
  F[profile_has_access]
end
S[Sessão user_profile_id] --> G
S --> C
S --> K
S --> R
F --> G
F --> C
F --> R
F --> P

```

3. Instalação e pré-requisitos (uma vez)

Execute no **SQL Editor do Supabase**, nesta ordem:

Ordem	Script	O que faz
1	scripts/access-control-schema.sql	Tabelas, funções, seed inicial
2	scripts/access-control-profile-write-rpc.sql	ACL nas RPCs de perfil (9d)
3	scripts/access-control-admin-rpc.sql	UI admin de papéis/grants (9e)
4	scripts/access-control-table-rls.sql	RLS nas tabelas principais (9f)

Scripts **incrementais** (se o seed for antigo ou algo sumir no app):

Script	Quando usar
access-control-member-dashboard-grants.sql	Cards do dashboard não aparecem para member
access-control-member-profile-columns.sql	CPF ou alertas alimentares não aparecem em Dados cadastrais
access-control-lider-escala.sql	Papel Líder, vínculo perfil↔tipo de escala e enforcement nas RPCs de escala
access-control-role-display-order.sql	Corrigir ordem dos papéis no painel (Visitantes → ... → Super administrador)
access-control-congregado-visitantes-roles.sql	Congregado e Visitantes não aparecem na UI — cria ambos os papéis e grants
access-control-congregado-role.sql	Só Congregado (legado; prefira o script combinado acima)

3.1 Primeiro super administrador

Todo ambiente precisa de **pelo menos um** perfil com papel `super_admin` :

```
-- Confirme quem já é super_admin
select p.full_name, p.phone, ar.code
  from public.profile_access_roles par
 join public.access_roles ar on ar.id = par.role_id
 join public.profiles p on p.id = par.profile_id
 where ar.code = 'super_admin';

-- Atribuir (troque o telefone)
insert into public.profile_access_roles (profile_id, role_id)
select p.id, ar.id
  from public.profiles p
 cross join public.access_roles ar
 where regexp_replace(coalesce(p.phone, ''), '\D', '', 'g')
        = regexp_replace('(11) 99999-9999', '\D', '', 'g')
        and ar.code = 'super_admin'
 on conflict (profile_id, role_id) do nothing;
```

3.2 Garantir `member` em todos os perfis

```
insert into public.profile_access_roles (profile_id, role_id)
select p.id, ar.id
  from public.profiles p
 cross join public.access_roles ar
 where ar.code = 'member'
        and not exists (
          select 1
            from public.profile_access_roles par
           where par.profile_id = p.id
                 and par.role_id = ar.id
        );
```

4. Operação pelo aplicativo (recomendado)

4.1 Quem pode abrir a manutenção de ACL

1. Perfil com papel `super_admin` .

2. Acesso à tela **Manutenção** (`/maintenance-dashboard`) — normalmente só `super_admin` ou quem tiver grant explícito nessa tela.
3. No carrossel de manutenção, o card **Controle de Acesso** só aparece para `super_admin` .

4.2 Aba Perfis — atribuir papéis a uma pessoa

Objetivo: definir *quem* é membro, quem cuida de eventos, quem é admin, etc.

Passo a passo:

1. Abra **Manutenção** → **Controle de Acesso** → **aba Perfis**.
2. Em **Buscar perfil**, digite nome, telefone ou código do membro (mínimo 2 caracteres).
3. Toque em **Buscar** e selecione o perfil na lista.
4. Na lista de papéis, use o **switch** para ligar ou desligar cada papel.
5. Confirme o toast de sucesso.

Efeito: alteração imediata em `profile_access_roles` .

Regras de segurança:

- O sistema **impede remover o último** `super_admin` do banco.
- Após mudar papéis de **si mesmo** ou de quem está testando, peça **Sair** → **entrar de novo** no app.

4.3 Aba Papéis — ajustar o que cada papel pode fazer

Objetivo: definir *o que* `member` , `events_admin` , etc. podem ver e editar.

Passo a passo:

1. Aba **Papéis**.
2. Selecione o papel (ex.: `member` , `events_admin`).
3. Filtre por **Telas**, **Tabelas** ou **Colunas**.
4. Para cada recurso, use:
 - **Ver** — `can_view`
 - **Editar** — `can_update` (só fica ativo se **Ver** estiver ligado)
5. Colunas sensíveis (`profiles.cpf` , `profiles.access_pin`) aparecem **destacadas em amarelo**.

Desligar Ver e Editar remove o grant daquele recurso para o papel.

Exemplos de política:

Objetivo	Onde ajustar
Membro não vê PIN	Papel member → Colunas → <code>profiles.access_pin</code> → Ver/Editar desligados
Só tesouraria vê financeiro	Tirar <code>dashboard.card.financial</code> e <code>/financial</code> do member; criar papel <code>finance_admin</code> (futuro) ou grant direto ao perfil

Equipe de eventos mantém agenda	Atribuir papel events_admin à pessoa (aba Perfis); revisar grants do papel em Telas/Tabelas
---------------------------------	---

4.4 O que o membro comum experiencia (sem ser admin)

Área	Comportamento
Dashboard	Só cards com can_view no papel
Dados cadastrais	Só campos com grant de coluna; sem PIN se não houver grant
Telas (pastoral, família, financeiro)	Guard bloqueia com alerta se não houver screen:* view
Gravação de perfil	RPC + RLS validam de novo no servidor

5. Operação via SQL (Table Editor ou SQL Editor)

Use quando a UI não estiver disponível, para diagnóstico ou carga em massa.

5.1 Consultar papéis de um perfil

```
select p.full_name, p.phone, ar.code, ar.name, par.granted_at
  from public.profiles p
 join public.profile_access_roles par on par.profile_id = p.id
 join public.access_roles ar on ar.id = par.role_id
 where regexp_replace(coalesce(p.phone, ''), '\D', '', 'g')
        = regexp_replace('(11) 99999-9999', '\D', '', 'g')
 order by ar.code;
```

5.2 Testar permissão (simula o app)

```
select public.profile_has_access(
  '<uuid-do-perfil>::uuid',
  'screen',           -- ou 'table' ou 'column'
  '/financeiro',     -- ou 'profiles' ou 'profiles.cpf'
  'view'             -- ou 'update'
);
```

Simular header da sessão (RLS):

```
select set_config(
  'request.headers',
  '{"x-profile-id":"<uuid-do-perfil>"}',
```

```
    true
);
select public.session_has_resource_access('table', 'profiles', 'view');
```

5.3 Atribuir / remover papel

```
-- Atribuir events_admin
insert into public.profile_access_roles (profile_id, role_id)
select p.id, ar.id
  from public.profiles p
 cross join public.access_roles ar
 where p.id = '<uuid> '::uuid
       and ar.code = 'events_admin'
on conflict (profile_id, role_id) do nothing;

-- Remover events_admin
delete from public.profile_access_roles par
 using public.access_roles ar
 where par.role_id = ar.id
       and par.profile_id = '<uuid> '::uuid
       and ar.code = 'events_admin';
```

5.4 Ajustar grant de um papel

```
-- Ex.: member pode VER mas não EDITAR financeiro (tela)
insert into public.access_grants (role_id, resource_id, can_view, can_update)
select r.id, res.id, true, false
  from public.access_roles r
 join public.access_resources res
    on res.resource_type = 'screen'
    and res.resource_key = '/financial'
 where r.code = 'member'
on conflict (role_id, resource_id) where (role_id is not null) do update
 set can_view = excluded.can_view,
     can_update = excluded.can_update,
     updated_at = now();

-- Remover grant (desliga view e update)
delete from public.access_grants g
 using public.access_roles r, public.access_resources res
 where g.role_id = r.id
       and g.resource_id = res.id
       and r.code = 'member'
```



```
and res.resource_type = 'screen'
and res.resource_key = '/financial';
```

5.5 Grant excepcional a um perfil (sem mudar o papel)

Quando **uma pessoa** precisa de algo que o papel dela não tem:

```
insert into public.access_grants (profile_id, resource_id, can_view, can_update)
select '<uuid-perfil> '::uuid, res.id, true, false
  from public.access_resources res
 where res.resource_type = 'screen'
       and res.resource_key = '/maintenance-dashboard'
on conflict (profile_id, resource_id) where (profile_id is not null) do update
 set can_view = excluded.can_view,
    can_update = excluded.can_update,
    updated_at = now();
```

Use com moderação: grants por perfil são mais difíceis de auditar que grants por papel.

6. Cenários operacionais (receitas)

6.1 Atribuir papel Congregado (em vez de Membro)

Use para quem participa do app mas **não** é membro pleno nem responsável familiar.

1. Controle de Acesso → **Perfis** → buscar pessoa.
2. Ligar **congregado** ; desligar **member** se não for membro pleno.
3. Ajustar grants em **Papéis** → **Congregado** (Telas / Colunas) conforme a política da igreja.
4. Pedir **Sair** → **entrar** no app.

Diferença padrão vs member : sem `/manage-members` , sem card lista de membros, sem financeiro, sem CPF/alertas alimentares no seed inicial.

6.2 Novo membro cadastrado no app

Etapa	O que acontece	Ação do operador
Cadastro	Cria linha em profiles	Nenhuma, se o seed aplicar member a todos automaticamente
Primeiro login	PIN + sessão	Nenhuma
Permissões	Herda grants do papel member	Revisar se a política da igreja está correta no papel member

Se o membro não tiver papel `member` :

```
insert into public.profile_access_roles (profile_id, role_id)
select p.id, ar.id
  from public.profiles p
 cross join public.access_roles ar
 where p.id = '<uuid> '::uuid
       and ar.code = 'member'
on conflict do nothing;
```

6.3 Nomear responsável pela equipe de eventos

1. **App**: Controle de Acesso → Perfis → buscar pessoa → ligar `events_admin` .
2. **Verificar** grants do papel `events_admin` (aba Papéis → Telas: `/maintenance-dashboard` ; Tabelas: `events` , `event_registrations`).
3. Pedir à pessoa: **Sair** → **entrar**.
4. Confirmar: engrenagem de manutenção visível; consegue abrir Programação de Eventos.

6.4 Restringir CPF e PIN apenas a administradores

1. Aba **Papéis** → `member` → **Colunas**.
2. Desligar **Ver** e **Editar** em:
 - `profiles.cpf`
 - `profiles.access_pin`
3. Manter ligado em `super_admin` (via curinga `*` ou grants explícitos).
4. Testar com perfil `member` : Dados cadastrais sem CPF/PIN; alteração de PIN falha no servidor.

6.5 Membro perdeu acesso a um card do dashboard

Diagnóstico:

```
select public.profile_has_access(
  '<uuid> '::uuid,
  'screen',
  'dashboard.card.financial',
  'view'
);
```

Correção rápida: executar `access-control-member-dashboard-grants.sql` ou religar o grant na aba Papéis → `member` → Telas.

6.6 Promover novo super administrador

1. Controle de Acesso → Perfis → buscar pessoa → ligar `super_admin` .
2. **Não** remover o `super_admin` antigo até o novo confirmar acesso.
3. Novo admin: Sair → entrar → abrir Manutenção → Controle de Acesso.

6.7 Desligar acesso de um voluntário

1. Remover papéis específicos (`events_admin` , `pastoral` , ...) na aba Perfis.
 2. Manter `member` se a pessoa continuar usando o app como membro.
 3. Para bloqueio total: remover todos os papéis **exceto** se for o último `super_admin` (bloqueado pelo sistema).
-

7. Rotina de manutenção recomendada

7.1 Semanal (5 min)

- ☐ Confirmar que existe pelo menos um `super_admin` ativo.
- ☐ Revisar se há perfis sem papel `member` (consulta SQL abaixo).

```
select p.id, p.full_name, p.phone
  from public.profiles p
 where not exists (
   select 1
     from public.profile_access_roles par
    join public.access_roles ar on ar.id = par.role_id
   where par.profile_id = p.id
     and ar.code = 'member'
 )
 and coalesce(p.full_name, '') <> '';
```

7.2 Ao mudar equipe (eventos, pastoral, tesouraria)

- ☐ Atribuir/remover papéis na aba Perfis.
- ☐ Avisar: **Sair e entrar** no app.
- ☐ Testar uma tela e um card do dashboard com o perfil afetado.

7.3 Após deploy de nova versão do app

- ☐ Confirmar scripts SQL do ACL aplicados (seção 3).
- ☐ Login como `member` e como `super_admin` .
- ☐ Um teste de gravação em Dados cadastrais e um de manutenção (se aplicável).

7.4 Ao alterar política de privacidade

- ☐ Revisar colunas `profiles.cpf`, `profiles.medical_food_alerts`, `profiles.access_pin` no papel `member`.
 - ☐ Documentar internamente quem pode ver dados sensíveis.
-

8. Testes de validação

8.1 Checklist rápido no celular

#	Perfil	Ação	Resultado esperado
1	member	Abrir dashboard	Cards conforme grants do member
2	member	Dados cadastrais	Campos básicos editáveis; sem seção PIN
3	member	Abrir Manutenção	Negado (sem engrenagem / sem acesso)
4	super_admin	Controle de Acesso	Card visível; busca e switches funcionam
5	events_admin	Manutenção → Eventos	Consegue criar/editar evento
6	Qualquer	Após mudança de papel	Sair → entrar → comportamento atualizado

8.2 Testes SQL úteis

```
-- ACL está ativo?
select public.acl_enforcement_enabled();

-- Últimos grants do member (amostra)
select ar.code, res.resource_type, res.resource_key, g.can_view, g.can_update
  from public.access_grants g
  join public.access_roles ar on ar.id = g.role_id
  join public.access_resources res on res.id = g.resource_id
 where ar.code = 'member'
 order by res.resource_type, res.resource_key
 limit 30;

-- PIN bloqueado para member (RPC deve falhar)
select public.update_profile_access_pin('(11) 99999-9999', '1234', '5678');
-- esperado: exception de permissão
```

9. Solução de problemas

Sintoma	Causa provável	Correção
Card sumiu do dashboard	Grant <code>dashboard.card.*</code> ausente no member	<code>access-control-member-dashboard-grants.sql</code> ou aba Papéis
“Acesso negado” ao abrir tela	Sem <code>screen:... view</code>	Aba Papéis ou grant SQL
Campo não aparece em Dados cadastrais	Sem <code>column:profiles.<campo> view</code>	<code>access-control-member-profile-columns.sql</code> ou aba Papéis → Colunas
Edição falha com mensagem de permissão	Sem <code>can_update</code> na coluna ou RPC	Aba Papéis; conferir script 9d aplicado
Mudança de papel não surte efeito	Sessão antiga	Sair → entrar
Controle de Acesso não aparece	Perfil não é <code>super_admin</code>	Atribuir papel (seção 3.1)
Erro ao abrir Controle de Acesso	RPC admin não instalada	Executar <code>access-control-admin-rpc.sql</code>
SELECT/UPDATE falha com RLS	<code>access-control-table-rls.sql</code> não aplicado ou sem header	Script 9f; app atualizado com <code>supabaseSessionFetch</code>
Tudo liberado indevidamente	Nenhum grant em <code>access_grants</code> (modo legado)	Executar seed em <code>access-control-schema.sql</code>

9.1 Modo legado

Se a tabela `access_grants` estiver **vazia**, `profile_has_access` retorna **true** para tudo (compatibilidade). Assim que existir **qualquer** grant, o ACL passa a valer de forma restritiva.

10. Boas práticas e limitações

10.1 Boas práticas

1. Prefira mudanças por **papel**, não por perfil individual (mais fácil de manter).
2. Conceda `super_admin` só a pessoas de confiança (2–3 no máximo).
3. Colunas **críticas** (`access_pin` , `cpf`) só para papéis administrativos.
4. Sempre peça **Sair → entrar** após alterar papéis.
5. Teste com um perfil “voluntário” antes de aplicar em massa.

10.2 O que o ACL ainda não cobre totalmente

Área	Situação
------	----------

RPCs de escalas	Com ACL por tipo quando <code>access-control-lider-escala.sql</code> está instalado; financeiro em lote ainda parcial
Tabelas checkins, <code>escalas_*</code>	RLS legada em parte das tabelas; operações sensíveis preferem <code>RPC security definer</code>
Mascaramento de CPF/PIN no SELECT	Coluna ainda pode existir no JSON se grant de tabela for amplo
Editar perfil de outra pessoa pela UI	Não há tela admin de cadastro; só SQL ou futura feature
Login / cadastro	Fluxos sem <code>x-profile-id</code> tratados como pré-sessão

11. Referência rápida de arquivos

Arquivo	Função
<code>MANUAL_CONTROLE_ACESSO.md</code>	Este manual
<code>CONTROLE_ACESSO.md</code>	Modelo técnico e inventário
<code>scripts/access-control-schema.sql</code>	Base do ACL
<code>scripts/access-control-admin-rpc.sql</code>	API da UI admin
<code>scripts/access-control-profile-write-rpc.sql</code>	RPCs de perfil com ACL
<code>scripts/access-control-table-rls.sql</code>	RLS por tabela
<code>scripts/access-control-member-dashboard-grants.sql</code>	Correção cards member
<code>scripts/access-control-member-profile-columns.sql</code>	Correção colunas member
<code>components/MaintenanceAccessControlCard.tsx</code>	UI no app
<code>hooks/useScreenAccessGuard.ts</code>	Bloqueio de rotas
<code>lib/accessControl.ts</code>	Cliente <code>profile_has_access</code>
<code>lib/supabaseSessionFetch.ts</code>	Header <code>x-profile-id</code>

12. Contatos e escalação

Situação	Responsável
Atribuição de papéis do dia a dia	Super administrador da igreja

Política de quem vê CPF/dados de saúde	Liderança + secretaria
Script SQL, deploy, bug no app	Equipe de desenvolvimento / TI
Perda do último super_admin	Recuperação via SQL Editor (service role) com insert manual em profile_access_roles

Última atualização: maio/2026 — alinhado às fases 9a–9f e guards de rota.

Parte 2 — Controle de acesso: modelo e inventário

Controle de acesso — modelo e inventário

Este documento rastreia **como o app-igreja identifica usuários hoje**, quais **telas, tabelas e campos** existem no ecossistema, e como a tabela de relacionamento proposta se encaixa.

Script SQL: [scripts/access-control-schema.sql](#)

Manual operacional (dia a dia): [MANUAL_CONTROLE_ACESSO.md](#)

Pacote: [PACOTE_3_GOVERNANCA_TI.md](#) · Índice: [INDICE_DOCUMENTACAO.md](#)

Status da implementação (atualizado em 22/05/2026)

Documento de encerramento da sessão: o que já está pronto, o que falta e qual é o **próximo passo** recomendado.

Concluído no Supabase (Passos 1–7)

Item	Status
Tabelas access_resources, access_roles, access_grants, profile_access_roles	Criadas
Correção de índices únicos (role_id / profile_id parciais)	Aplicada (evita erro 23505 com profile_id null)
Funções profile_has_access e profile_has_access_by_phone	Criadas
Seed de recursos, papéis e grants (member, super_admin, events_admin)	Executado

Perfil administrador (04b919ba-38b4-4fe5-a371-2e98e9acbc0d) com super_admin + member	Configurado
Todos os perfis em profiles com papel member	Configurado
Teste SQL: profile_has_access → manutenção true para super_admin	OK

Concluído no app (Passo 9)

Item	Arquivo(s)
Sessão: user_phone + user_profile_id após login/cadastro	lib/userSession.ts, app/index.tsx, app/register.tsx
Cliente ACL: RPC profile_has_access / sessionHasAccess	lib/accessControl.ts
Engrenagem do dashboard só para quem tem view em /maintenance-dashboard	app/(tabs)/dashboard.tsx
Bloqueio da tela maintenance-dashboard sem permissão	app/maintenance-dashboard.tsx
Logout limpa sessão (telefone + profile_id)	app/(tabs)/dashboard.tsx
Passo 9b: carrossel só com cards permitidos (dashboard.card.*)	lib/accessControl.ts, app/(tabs)/dashboard.tsx
Passo 9c: colunas visíveis/editáveis em Dados cadastrais	lib/accessControl.ts, app/manage-profile.tsx
Passo 9d: update_profile_field e update_profile_access_pin validam can_update	scripts/access-control-profile-write-rpc.sql
Passo 9e: UI admin de papéis e grants (somente super_admin)	components/MaintenanceAccessControlCard.tsx, scripts/access-control-admin-rpc.sql
Passo 9f: RLS nas tabelas com profile_has_access + header x-profile-id	scripts/access-control-table-rls.sql, lib/supabaseSessionFetch.ts

Ainda não feito (próximas sessões)

- Enforcement ACL em RPCs auxiliares restantes (financeiro em lote, etc.). Escalas: feito em `access-control-lider-escala.sql` .
- Papel `events_admin` atribuído a pessoas da equipe de eventos (só seed SQL hoje).
- RLS em tabelas auxiliares (`checkins` , `escalas_*` , ...).

Outras entregas da mesma época (fora do ACL)

- Senha de acesso em Dados cadastrais (PIN, seção recolhível, olho, validação).
- Gerenciar Família: seção “Adicionar membro” recolhível.
- Dashboard: botões < / > avançam 1 card a cada 500 ms enquanto pressionados.

Próximo passo recomendado (quando retomar)

Roadmap ACL (fases 9a–9f) **concluído**. Guards de rota nas telas principais via `hooks/useScreenAccessGuard.ts` .

Próximas melhorias opcionais:

1. Enforcement ACL em RPCs de manutenção (escalas, financeiro em lote, etc.).
2. RLS em tabelas auxiliares (`checkins` , `escalas_log` , `tipos_escala` , ...).
3. Views com mascaramento de colunas sensíveis (`access_pin` , `cpf`) em `profiles` .

Supabase (obrigatório após 9f): execute `scripts/access-control-table-rls.sql` no SQL Editor.

Teste rápido (9f):

```
-- Simula header do app (substitua pelo seu profile_id)
select set_config('request.headers', '{"x-profile-id":"<uuid>"}', true);
select public.session_has_resource_access('table', 'profiles', 'view');
-- esperado: true para member com grant
```

Lembrete ao testar no celular: após mudar papéis no Supabase, use **Sair** e entre de novo para gravar `user_profile_id` na sessão.

1. Situação no banco vs. no app (referência)

Aspecto	Hoje
Identidade no app	<code>profiles.id</code> (login por PIN em <code>profiles.access_pin</code> + telefone em <code>AsyncStorage</code>)
Supabase Auth	Opcional (<code>profiles.auth_user_id</code>); muitos usuários não têm linha em <code>auth.users</code>
Autorização	RLS com <code>profile_has_access</code> nas tabelas principais (9f) + RPCs <code>security</code> definir para fluxos sem sessão
Manutenção	Engrenagem e rota protegidas via <code>profile_has_access</code> (Passo 9 parcial)

Cards do dashboard	Filtrados por <code>profile_has_access</code> (Passo 9b)
Demais telas	Guards de rota em manutenção, perfil, família, pastoral, financeiro e LGPD
Campos sensíveis	Ocultos no cliente; RPCs <code>update_profile_field</code> / PIN validam <code>can_update</code> por coluna (9d)

O banco **já responde** permissões via `profile_has_access` ; o app consulta para **manutenção e cards do dashboard**.

2. Sujeito da permissão

Use sempre `profiles.id` como "usuário" do ACL.

- Não use só `auth.users.id` — pedidos pastorais e RPCs já documentam o desvio (`pastoral-requests-fields.sql`).
- O app pode resolver `profile_id` a partir do telefone da sessão (`find_profile_id_by_phone` / SELECT em `profiles`).

3. Inventário de telas (`resource_type = 'screen'`)

resource_key	Rota / origem	Observação
<code>screen:/</code>	<code>app/index.tsx</code>	Login
<code>screen:/register</code>	<code>app/register.tsx</code>	Cadastro
<code>screen:/dashboard</code>	<code>app/(tabs)/dashboard.tsx</code>	Painel principal
<code>screen:/maintenance-dashboard</code>	<code>app/maintenance-dashboard.tsx</code>	Manutenção (eventos, monitor salas)
<code>screen:/manage-profile</code>	<code>app/manage-profile.tsx</code>	Dados cadastrais
<code>screen:/manage-members</code>	<code>app/manage-members.tsx</code>	Gerenciar família
<code>screen:/pastoral</code>	<code>app/pastoral.tsx</code>	Coração Aberto (formulário)
<code>screen:/pastoral-history</code>	<code>app/pastoral-history.tsx</code>	Meus pedidos
<code>screen:/lgpd</code>	<code>app/lgpd.tsx</code>	Termos LGPD

Cards do dashboard (`screen:dashboard.card.*`)

resource_key	Card	content
--------------	------	---------

screen:dashboard.card.event_alt	Agenda da Família	event_alt
screen:dashboard.card.qr	Check In	qr
screen:dashboard.card.kids_teens	SALA(S)	kids_teens
screen:dashboard.card.offerings	Dízimos e Ofertas	offerings
screen:dashboard.card.pastoral	Coração Aberto	pastoral
screen:dashboard.card.members_list	Lista de membros	members_list
screen:dashboard.card.birthdays	Aniversariantes	birthdays
screen:dashboard.card.vigilance_scales	Escalas	vigilance_scales
screen:dashboard.card.parking_vehicle_v2	Estacionamento	parking_vehicle_v2
screen:dashboard.card.grouped_manage	Menu (perfil + família)	grouped_manage

Visibilidade condicional de cards (parâmetros/evento) continua no app; o ACL define se o usuário **pode** ver o card quando ele estaria disponível.

4. Inventário de tabelas (resource_type = 'table')

Tabelas usadas pelo app (Supabase public):

resource_key	Uso principal
table:profiles	Login, perfil, LGPD, endereço, PIN
table:members	Família, check-in, listas
table:events	Agenda, manutenção, salas
table:event_registrations	Check-in / salas Kids-Teens
table:profile_vehicles	Veículos no perfil e estacionamento
table:pastoral_requests	Pedidos pastorais
table:pastoral_reason_categories	Motivos (leitura)
table:pastoral_reason_subcategories	Submotivos (leitura)
table:app_parameters	Parâmetros globais (PIX, QR, prefixo família)
table:families	Dados de família (useFamilyData)
table:vigilancia_*	Escalas (import/histórico — conferir nomes no Supabase)

Atualizar a lista após `information_schema.tables` no projeto se houver tabelas só no banco.

5. Inventário de campos (`resource_type = 'column'`)

Formato: `column:<tabela>.<coluna>` .

profiles (cadastro + sensíveis)

Colunas conhecidas no app (`manage-profile.tsx` , `register.tsx`):

Coluna	Sensível	Notas
full_name, phone, birth_date, email	médio	Contato / identificação
cpf	alto	Oculto na UI padrão
access_pin	crítico	Só RPC <code>update_profile_access_pin</code> ; exige <code>can_update</code> (9d)
address_*, cep	médio	Endereço
lgpd_*, medical_food_alerts	alto	Privacidade / saúde
family_id, codigo_membro, role	médio	Escopo familiar / papel
selfie_url	médio	Imagem
auth_user_id, id, created_at, updated_at	sistema	Bloqueados em <code>update_profile_field</code>

members

Coluna	Notas
full_name, phone, birth_date, relationship, family_id	Gerenciar família
accepted	Reconhecimento na família

events

Campos editáveis em `maintenance-dashboard` / `maintenanceEventForm` : `name` , `event_date` , `event_local` , `max_capacity` , `kids_room` , `teens_room` , `parm_ofertas` , `is_locked` , `is_visible` , etc.

pastoral_requests

request_for , beneficiary_* , destination_label , profile_id , message , status, etc. (pastoral-requests-fields.sql).

Regra sugerida: can_view na tabela não implica todas as colunas — conceda colunas sensíveis (cpf , access_pin) só a papéis administrativos.

6. Modelo proposto (3 tabelas + função)

```
erDiagram
    profiles ||--o{ profile_access_roles : tem
    access_roles ||--o{ profile_access_roles : agrupa
    access_roles ||--o{ access_grants : recebe
    profiles ||--o{ access_grants : direto
    access_resources ||--o{ access_grants : alvo

    access_resources {
        uuid id PK
        text resource_type
        text resource_key
        text label
    }
    access_roles {
        uuid id PK
        text code UK
        text name
    }
    access_grants {
        uuid id PK
        uuid role_id FK
        uuid profile_id FK
        uuid resource_id FK
        boolean can_view
        boolean can_update
    }
```

access_resources (catálogo)

Define **o que** pode ser protegido: tela, tabela ou coluna.

access_roles (papéis)

Ex.: member , family_acceptor , pastoral , events_admin , super_admin .

access_grants (relacionamento pedido)

Uma linha = permissão de um **papel** ou de um **perfil** sobre um recurso:

- can_view — visualizar (tela, listagem, SELECT de campo)
- can_update — alterar (formulário, UPDATE, RPC de escrita)

Exatamente um de role_id ou profile_id deve estar preenchido.

profile_access_roles

N:N entre profiles e access_roles .

Função profile_has_access(profile_id, resource_type, resource_key, action)

- action : 'view' ou 'update'
- Suporta curinga * no final da chave (ex.: table:profiles não inclui colunas; column:profiles.* todas as colunas)
- **Modo legado:** se não existir nenhum grant no sistema, retorna true (app continua funcionando até você configurar papéis)

7. Papéis sugeridos (seed)

code	Quem	View típico	Update típico
visitantes	Sem perfil na sessão ou perfil sem papéis	Login, cadastro, LGPD, check-in QR, eventos públicos	Cadastro e inscrição em eventos
congregado	Participante cadastrado	Dashboard básico, perfil, pastoral; sem família/financeiro	Perfil e pedido pastoral
member	Membro comum	Próprio perfil (campos básicos), cards dashboard, pastoral próprio	Perfil próprio (campos permitidos), pedido pastoral
family_acceptor	Quem aceita familiares	manage-members, membros da família	members da própria family_id

lider	Líder de tipo(s) de escala	Painéis de servos/programação + card Escalas	Tipos vinculados em profile_scale_leadership
events_admin	Equipe de eventos	maintenance-dashboard, events	CRUD events
pastoral	Equipe pastoral	Pedidos (futuro painel)	Triagem pastoral_requests
super_admin	TI / pastor responsável	*	*

Ordem no painel admin: mesma sequência da tabela acima.

Ajuste conforme a política da igreja.

8. Roadmap do app (ordem sugerida)

Fase	Descrição	Status
9a	Manutenção (engrenagem + tela)	Feito
9b	Cards do dashboard (dashboard.card.*)	Feito
9c	Campos do perfil (column:profiles.*)	Feito
9d	RPCs de escrita	Feito
9e	UI admin de grants	Feito
9f	RLS nas tabelas	Feito

9. Checklist operacional

Supabase

- ☒ Executar `scripts/access-control-schema.sql` (e correção de índices se necessário)
- ☒ `super_admin` no perfil administrador
- ☒ `member` em todos os perfis
- ☒ Testar `profile_has_access` no SQL Editor
- ☐ Atribuir `events_admin` a quem cuida de eventos (quando definir a equipe)
- ☐ Revisar grants do papel `member` conforme política da igreja

App

- ☒ `lib/userSession.ts` + `lib/accessControl.ts`
-

- ☒ Login/cadastro persistem `user_profile_id`
- ☒ Dashboard: engrenagem condicional
- ☒ `maintenance-dashboard` : guard de acesso
- ☒ Dashboard: filtrar cards por ACL (Passo 9b)
- ☐ Supabase: `access-control-member-dashboard-grants.sql` se seed antigo omitiu cards do `member`
- ☒ `manage-profile` : colunas visíveis/editáveis por permissão (Passo 9c)
- ☐ Supabase: `access-control-member-profile-columns.sql` se seed antigo omitiu `cpf` / `medical_food_alerts` no `member`
- ☒ RPCs: validar `can_update` antes de gravar (Passo 9d)
- ☐ Supabase: `access-control-profile-write-rpc.sql` após deploy do app 9d
- ☒ Manutenção: card Controle de Acesso para `super_admin` (Passo 9e)
- ☐ Supabase: `access-control-admin-rpc.sql` após deploy do app 9e
- ☒ App: header `x-profile-id` em todas as requisições Supabase (Passo 9f)
- ☒ RLS: políticas ACL em tabelas principais (Passo 9f)
- ☐ Supabase: `access-control-table-rls.sql` após deploy do app 9f

Arquivos de referência no código

Arquivo	Uso
<code>lib/accessControl.ts</code>	Constantes <code>ACCESS_SCREEN</code> , <code>ACCESS_DASHBOARD_CARD</code> , helpers RPC
<code>lib/userSession.ts</code>	<code>user_profile_id</code> no <code>AsyncStorage</code>
<code>lib/supabaseSessionFetch.ts</code>	Header <code>x-profile-id</code> para RLS (9f)
<code>hooks/useScreenAccessGuard.ts</code>	Guard de rota por <code>screen:*</code>
<code>scripts/access-control-table-rls.sql</code>	Políticas RLS por tabela
<code>DASHBOARD_CARDS.md</code>	Lista de cards e content
<code>MANUTENCAO_ECOSISTEMA.md</code>	Rotina do módulo de manutenção
<code>MANUAL_CONTROLE_ACESSO.md</code>	Manual operacional (papéis, grants, testes, troubleshooting)

Parte 3 — Blueprint completo

Blueprint completo — app-igreja (Igreja Batista Norte)

Documento de referência da solução implementada: telas, controles, fluxos de negócio, mensagens ao usuário e camadas de segurança.

Índice da documentação: [INDICE_DOCUMENTACAO.md](#) · Pacote técnico: [PACOTE_3_GOVERNANCA_TI.md](#) · Anexo: [PACOTE_4_ANEXO_TECNICO.md](#)

Atualizado em: 22/05/2026

1. Visão geral da solução

Aspecto	Descrição
Produto	PWA / app Expo (React Native + Web) para membros e equipe da igreja
Backend	Supabase (PostgreSQL + PostgREST + RPCs + Storage)
Projeto Supabase	bldbrsuiwctoaxzcrjoc — URL https://bldbrsuiwctoaxzcrjoc.supabase.co
Identidade	Login por celular + PIN de 4 dígitos (<code>profiles.access_pin</code>); sessão local com <code>user_phone</code> e <code>user_profile_id</code>
Autorização	ACL via RPC <code>profile_has_access</code> + RLS com header <code>x-profile-id</code>
Deploy web	<code>npm run build:web</code> → pasta <code>dist/</code> (PWA estático em HTTPS)
Modo totem	Aparelho dedicado: celular configurado em <code>app_parameters.cel_totem</code> , senha fixa 9999, rota <code>/totem-checkin</code>

Mapa de telas (rotas)

<code>/</code>	→ Login
<code>/register</code>	→ Cadastro inicial
<code>/(tabs)/dashboard</code>	→ Painel principal (carrossel de módulos)
<code>/manage-profile</code>	→ Dados cadastrais
<code>/manage-members</code>	→ Gerenciar família
<code>/pastoral</code>	→ Coração Aberto (novo pedido)
<code>/pastoral-history</code>	→ Meus pedidos pastorais
<code>/financial</code>	→ Financeiro (leitura)
<code>/mapa-geolocalizacao</code>	→ Mapa de geolocalização (PWA/web)
<code>/lgpd</code>	→ Termos LGPD

/maintenance-dashboard → Manutenção (equipe)
/totem-checkin → Totem de check-in (quiosque)

Documentação relacionada

Documento	Conteúdo
CONTROLE_ACESSO.md	Modelo ACL, inventário de recursos, status de implementação
MANUAL_CONTROLE_ACESSO.md	Manual operacional do ACL
scripts/	Scripts SQL versionados para deploy no Supabase

2. Segurança da informação — camadas e confiança

2.1 Camadas de proteção (defesa em profundidade)

```
flowchart TB
    subgraph L1 [Camada 1 – Dispositivo]
        PIN[PIN 4 dígitos]
        AsyncStorage[Sessão local AsyncStorage]
        Camera[Permissões câmera]
        TotemIso[Totem isolado do fluxo membro]
    end
    subgraph L2 [Camada 2 – Cliente app]
        ScreenACL[Guard de tela sessionHasAccess]
        CardACL[Cards do dashboard filtrados por ACL]
        ColumnACL[Colunas do perfil view/update]
        Strict[EXPO_PUBLIC_ACL_STRICT fail-closed]
    end
    subgraph L3 [Camada 3 – Transporte]
        HTTPS[HTTPS Supabase]
        Header[x-profile-id em toda requisição]
        AnonKey[Chave anon – sem service_role no app]
    end
    subgraph L4 [Camada 4 – Servidor Supabase]
        RPC[RPCs security definer]
        RLS[Row Level Security]
        Grants[access_grants + profile_access_roles]
        NoDirect[INSERT escalas_log / PIN só via RPC]
    end
    L1 --> L2 --> L3 --> L4
```

2.2 Banco de dados e guarda de informações

Item	Detalhe
SGBD	PostgreSQL 15+ (gerenciado pelo Supabase)
Schema	public
Tabelas principais	profiles, members, events, event_registrations, checkins, financials, pastoral_requests, escalas_log, tipos_escala, access_resources, access_roles, access_grants, profile_access_roles, app_parameters, cep_geolocations, etc.
Dados sensíveis	access_pin (crítico), cpf, lgpd_*, medical_food_alerts — ocultos na UI padrão; escrita via RPC com can_update
Autenticação Supabase Auth	Opcional (profiles.auth_user_id); maioria dos usuários não usa auth.users
Validação de login	RPC verificar_login — PIN nunca comparado em texto claro no cliente
Sessão no app	user_profile_id + user_phone em AsyncStorage; reparada via repairUserSessionReference() se inconsistente
RLS	Políticas consultam profile_has_access + header x-profile-id injetado por lib/supabaseSessionFetch.ts
Modo estrito ACL	EXPO_PUBLIC_ACL_STRICT=true em produção: nega acesso se RPC ACL ausente (banner no dashboard)
Totem	Sem ACL de tela; confiança no aparelho físico + parâmetro cel_totem + PIN 9999; confirmação só via RPC com pré-check-in
Geolocalização	CEP → coordenadas no servidor (cep_geolocations); cache local versionado (geoCepCache.v8)
LGPD	Aceite registrado em profiles.lgpd_accepted; termos carregados de app_parameters; scroll obrigatório antes do aceite
Confiança operacional	Dados em nuvem Supabase (SOC 2); app não embute service_role; scripts SQL versionados em scripts/; PWA servido em HTTPS

2.3 Papéis de acesso (ACL)

Papéis canônicos (ordem de exibição em `lib/accessRoleDisplayOrder.ts`):

visitantes → congregado → member → family_acceptor → lider → events_admin → pastoral → super_admin

Recursos protegidos:

- **Telas** — `screen:/dashboard`, `screen:/manage-profile`, etc.

- **Cards** — `screen:dashboard.card.*`
- **Tabelas** — `table:profiles` , `table:members` , etc.
- **Colunas** — `column:profiles.cpf` , `column:profiles.access_pin` , etc.

2.4 Matriz de guards por tela

Tela	Mecanismo	Resource key
Login /	Público	—
Cadastro /register	Público (com ?phone=)	—
Dashboard	Cards filtrados por ACL	<code>screen:/dashboard + dashboard.card.*</code>
Dados cadastrais	<code>sessionHasAccess</code> manual	<code>/manage-profile</code>
Gerenciar família	<code>useScreenAccessGuard</code>	<code>/manage-members</code>
Coração Aberto	<code>useScreenAccessGuard</code>	<code>/pastoral</code>
Meus pedidos	<code>useScreenAccessGuard</code>	<code>/pastoral-history</code>
Financeiro	<code>useScreenAccessGuard</code>	<code>/financial</code>
Mapa (web)	<code>useScreenAccessGuard</code>	<code>/mapa-geolocalizacao</code>
LGPD	<code>useScreenAccessGuard</code> (skip com ?phone=)	<code>/lgpd</code>
Manutenção	<code>useScreenAccessGuard</code>	<code>/maintenance-dashboard</code>
Totem	Sem ACL — aparelho dedicado	—

3. Telas — descrição completa

3.1 Tela de Login (/ — `app/index.tsx`)

Esta tela serve para: autenticar membros (celular + PIN) ou abrir o modo totem em aparelho dedicado; restaurar sessão existente; links para redes sociais.

Modo membro (padrão)

Elemento	Função
Logo IBNORTE	Identidade visual
Título "Boas-Vindas"	Cabeçalho

Subtítulo	Orienta: celular → WhatsApp → PIN temporário → alterar em Dados Cadastrais
Campo Celular	Máscara (00) 00000-0000; ao completar 11 dígitos foca o PIN
Botão X (celular)	Limpa o número digitado
Campo Senha de acesso	4 dígitos, ocultos; auto-submit ao completar
Ícone WhatsApp	Gera e envia PIN temporário (fluxo primeira entrada)
Texto de hint (PIN)	Varia conforme psw_user/psw_mngr em app_parameters
Botão "Entrar"	Valida e autentica
Instagram / YouTube	Abre links externos da igreja

Modo totem (quando celular = `cel_totem`)

Elemento	Função
Título "Totem — Check-in"	Identifica modo quiosque
Campo Senha do totem	Apenas PIN; sem campo celular
Botão "Abrir tela do totem"	Senha 9999 → /totem-checkin
Hint	Informa que não usa cadastro/LGPD/PIN de membro

Restauração de sessão

- Ao abrir: se há `user_phone` salvo e é o totem → redireciona direto para `/totem-checkin` .
- Parâmetro `?signedOut=1` após logout impede restauração automática.

Mensagens interativas (Alert)

Título	Quando
Atenção — celular inválido	WhatsApp ou entrar sem celular completo
WhatsApp indisponível	psw_mngr/psw_user mal configurado
Erro ao gerar código	Falha em prepareAccessPinDraft
Gestor não configurado	psw_user=nao sem psw_mngr
Não foi possível gerar o código	Perfil/RPC ausente no Supabase
Código gerado	PIN preparado; mensagem copiada para área de transferência
Código necessário	Primeira entrada sem passar pelo WhatsApp
Senha incorreta	Totem: PIN diferente de 9999

Totem não configurado	cel_totem ausente
Validação indisponível	RPC verificar_login não instalada
Erro — Número ou senha inválidos	Credenciais incorretas
Erro de Acesso	Falha de rede ou perfil não continuável
Erro — link social	Falha ao abrir Instagram/YouTube

Quem inicia / termina

- **Inicia:** usuário digita celular e PIN (ou solicita PIN via WhatsApp).
- **Termina:** app grava sessão (`persistUserSession`) e redireciona para dashboard, cadastro, LGPD ou totem conforme estado do perfil.

3.2 Tela de Cadastro (`/register` — `app/register.tsx`)

Esta tela serve para: primeiro cadastro de perfil após receber PIN; coleta nome, nascimento, selfie, aceite LGPD; cria `profiles` + `family_id` .

Elemento	Função
Nome completo	Capitalização automática por palavra
Data nascimento	Máscara DD/MM/AAAA
Telefone	Somente leitura (vem da rota <code>?phone=</code>)
Caixa LGPD rolável	Termos carregados do banco; exige scroll até o fim
Checkbox "Li e aceito"	Marca <code>lgpd_accepted=true</code>
Checkbox "Li e não concordo"	Marca recusa (com alerta de privacidade)
Botão selfie / câmera	Abre captura (nativo) ou seletor de arquivo (web)
Estágio CAMERA	Preview frontal, botão "Capturar Selfie"
Estágio CONFIRM	Revisão da foto + confirmar cadastro
Botão final	Grava perfil, upload selfie no Storage, reserva <code>family_id</code>

Rejeição totem: `useRejectTotemPhoneFromMemberRoutes` redireciona celular totem para login.

Mensagens

Mensagem	Quando
Preencha Nome e Nascimento	LGPD sem formulário válido

Role os termos até o final	Aceite sem scroll completo
Privacidade (declínio LGPD)	buildLgpdDeclineMessage
Permissão necessária (câmera)	Selfie sem permissão
Erro na câmera	Falha de hardware
Sucesso — cadastro concluído	Redireciona para completar dados
Erro	Falha insert/update

Fluxo

- **Inicia:** gestor/sistema gera PIN → membro entra com PIN → rota de onboarding leva ao cadastro.
- **Termina:** perfil criado → sessão gravada → próxima tela (`manage-profile` ou LGPD).

3.3 Painel Principal / Dashboard (`/ (tabs)/dashboard`)

Esta tela serve para: hub central do membro — carrossel de módulos conforme evento selecionado e permissões ACL.

Cabeçalho global

Elemento	Função
"Boas-Vindas, {nome}"	Saudação; fundo vermelho se LGPD pendente
Título do card ativo	Nome do módulo atual
Banner ACL	ACL_UNAVAILABLE_MESSAGE se RPC ACL ausente em modo estrito

Rodapé

Elemento	Função
< / >	Navega cards (segurar = avança a cada 500 ms)
Menu	Volta ao índice do carrossel
Engrenagem	Manutenção — só com view em <code>/maintenance-dashboard</code> ; duplo toque evita clique acidental

Card 1 — Agenda da Família (`event_alt`)

Serve para: escolher evento e registrar **audiência** (pré-check-in) dos membros da família.

Elemento	Função
Evento selecionado	Nome, data/hora, local, badges Kids/Teens

Vagas	Inscritos / capacidade
Trocar Evento	Chips horizontais (FamilyEventSelector)
Lista de audiência	Checkboxes por membro (FamilyRegistrationList)
Checkbox em massa	Marca/desmarca todos (exceto quórum bloqueado)

Hints inline (sem Alert):

- Selecione um evento...
- Quórum: só membro da sessão ativa
- Quórum + totem confirmado: audiência travada
- Quórum pendente: marque audiência para liberar QR
- Check-in automático / QR só no dia do evento
- Erro de gate pré-check-in (`preCheckinGateError`)

Quem inicia / termina o pré-check-in:

- **Inicia:** membro marca checkbox na audiência.
- **Termina (totem/quórum):** membro apresenta QR no totem → RPC confirma → checkbox trava.
- **Staff:** não participa desta etapa; apenas configura evento na manutenção.

Card 2 — Check-in QR (`qr`)

Serve para: exibir etiqueta (código família) e QR para leitura no totem ou entrada manual.

Elemento	Função
Toque no card	Abre <code>CheckinModal</code> (seleção manual — ainda sem gravação no banco)
Etiqueta	<code>family_id</code> / <code>codigo_membro</code>
QR Code	Codifica identificador da família
Badges Kids/Teens	Se evento tem salas

Visibilidade: dia do evento + pré-check-in feito + ACL + tipo de fluxo (totem/quórum/manual).

Quem termina check-in no totem:

- **Membro** mostra QR na tela do **totem** (outro aparelho, `/totem-checkin`).
- **Totem** escaneia → `lookup_totem_checkin` → `confirm_totem_checkin` .
- **Sistema** atualiza `checkins.status` para `confirmado` .

Card 3 — SALA(S) (`kids_teens`)

Serve para: monitorar entrada nas salas Kids/Teens (somente leitura para o membro).

Elemento	Função
Chips IBN KIDS / IBN TEENS	Contagem check-in/total
Lista de inscritos	✓ se room_entry_checked

Quem termina check-in na sala:

- **Staff** marca entrada em **Manutenção** → **Sala(s) - Check In**.
- Membro apenas **visualiza** status aqui.

Card 4 — Dízimos e Ofertas (**offerings**)

Elemento	Função
Dados do recebedor	Informações institucionais
Chave PIX	Exibição + Copiar chave PIX
Atualizar chave PIX	Recarrega de app_parameters

Mensagens: Chave PIX indisponível; Erro ao copiar; sucesso inline 3 s.

Card 5 — Coração Aberto (**pastoral**)

Elemento	Função
Toque	Navega para /pastoral

Card 6 — Lista de Membros (**members_list**)

Elemento	Função
Mapa	/mapa-geolocalizacao
Busca	Filtra por nome
Tabela	Nome, família, WhatsApp
Ícone users	Modal "Membros da família"
Ícone Zap	Abre WhatsApp do membro

Card 7 — Aniversariantes (**birthdays**)

Elemento	Função
Seletor de mês	Picker
Lista	Data + nome + WhatsApp

Card 8 — Financeiro (`financial`)

Elemento	Função
Toque	<code>/financial</code> (somente leitura)

Card 9 — Escalas (`vigilance_scales` + `scale_roster`)

Elemento	Função
Lista de tipos	Radio → abre escala do tipo
Roster	Datas + servos + WhatsApp
Estacionamento	Se tipo = parking → painel de placa
Voltar	Retorna à lista de tipos

Card 10 — Dados Cadastrais (`grouped_manage`)

Elemento	Função
Dados Cadastrais	<code>/manage-profile</code>
Gerenciar Família	<code>/manage-members</code>

Modais do dashboard

CheckinModal: lista membros, Confirmar Presença / Cancelar — *implementação de persistência pendente.*

Modal família: lista membros, WhatsApp, Fechar.

3.4 Dados Cadastrais (`/manage-profile`)

Serve para: autogestão completa do perfil — dados pessoais, contato, endereço (CEP), selfie, veículos, vínculo familiar, PIN, LGPD.

Seção / controle	Função
Selfie	Captura/substituição com confirmação
Dados Pessoais	Nome, nascimento, CPF (se permitido) — edição inline
Contato	E-mail, telefone (com <code>changePhoneEverywhere</code>)
Endereço	CEP com sync automático de logradouro
Senha de acesso	PIN atual / novo / confirmar (4 dígitos)

Veículos	CRUD placa, marca, modelo, cor
Vincular família	Busca por código + solicitação
LGPD	Atalho se pendente
Voltar	Dashboard card grouped_manage

ACL: guard manual por tela; **colunas** filtradas por `canViewProfileColumn` / `canUpdateProfileColumn` ; campos bloqueados até ACL carregar.

Mensagens principais: Acesso negado; Complete seu cadastro (onboarding); Campo protegido; Senha atualizada; erros de CEP/câmera/veículo/família.

3.5 Gerenciar Família (/manage-members)

Serve para: CRUD de membros da família (`members`), reconhecimento familiar (aceite), herança de endereço.

Controle	Função
Formulário recolhível	Adicionar/editar membro
Busca por telefone	Vincula perfil existente
Parentesco	Chips (Cônjuge, Filho(a), etc.)
Checkbox aceite	Reconhecimento na família
Editar / Salvar / Excluir	Por membro
Banner family_id	Somente leitura

Regra: representante legal não pode ser excluído.

Mensagens: Acesso negado; duplicata de telefone/membro; confirmação de exclusão; RLS/accepted column errors.

3.6 Coração Aberto (/pastoral)

Serve para: enviar pedido de cuidado pastoral / intercessão.

Controle	Função
Motivo / Situação	Modais categoria/subcategoria
Beneficiário	Eu / família / terceiro (+ campos condicionais)
Destino	Sigilo pastoral / Intercessão

Seu pedido	Texto livre
Histórico (ícone)	/pastoral-history
Enviar pedido	RPC submitPastoralRequest

3.7 Meus Pedidos (/pastoral-history)

Serve para: listar pedidos do perfil logado com status; pull-to-refresh.

Controle	Função
Tentar novamente	Em erro de carga
Fazer um pedido / Novo pedido	Volta ao formulário

3.8 Financeiro (/financial)

Serve para: relatórios financeiros **somente leitura** — REALIZADO mensal, comparativo, 12 meses, planejado × realizado.

Controle	Função
Seletor de mês	Meses com REALIZADO e/ou PLANEJADO; badge (só planejado)
Hint só planejado	Explica resultado REALIZADO vazio
Resultado do mês	Boletim com saldo acumulado até o mês + YTD
Comparativo mensal	Mês atual vs anterior
Últimos 12 meses	Matriz
Planejado × Realizado	Bloqueado se sem orçamento planejado
Atualizar	Em erro de carga
Aviso amarelo	commentsWarning se comentários não carregaram

Edição financeira: apenas em **Manutenção** → **Informações Financeiras** (staff).

3.9 Mapa de Geolocalização (/mapa-geolocalizacao — web)

Serve para: mapa Leaflet com pins por CEP dos perfis; filtros visitante/membro.

Controle	Função
Filtros	Todos / Com papel / Visitantes
Atualizar mapa	Sincroniza snapshot + geocodificação

Pin clicável	Painel detalhe + WhatsApp
Voltar	Lista de membros no dashboard

Nativo (app mobile): placeholder informando que mapa é só na PWA.

3.10 Termos LGPD (/lgpd)

Serve para: exibir termos e registrar aceite/recusa em `profiles.lgpd_accepted`.

Controle	Função
Scroll obrigatório	Gate antes dos checkboxes
Li e aceito / não concordo	Escolha
Confirmar / Concluir	Grava preferência

ACL ignorado quando `?phone=` (fluxo cadastro).

3.11 Totem Check-in (/totem-checkin)

Serve para: quiosque — escanear QR da família e confirmar presença no evento do dia.

Controle	Função
Seleção automática de evento	Hoje + totem_ativo ou requer_quorum + publicado
Ativar câmera	Gate de permissão
Scanner QR	CameraView modo QR
Banner de status	Sucesso / erro / aviso
Encerrar sessão	Logout totem

Sem ACL de tela — modelo de aparelho dedicado.

Fluxo completo do check-in (totem/quórum):

1. STAFF cria evento (Manutenção) com totem e/ou quórum
2. MEMBRO marca audiência no Dashboard (pré-check-in)
3. MEMBRO abre card QR no dia do evento
4. TOTEM escaneia QR → lookup → confirm
5. SISTEMA grava confirmado; trava audiência se quórum
6. STAFF consulta Lista de Presença (Manutenção) – somente leitura

Mensagens (Toast + banners):

- Pré-check-in não encontrado
- Confirmação realizada com sucesso
- Já confirmado (TOTEM_CHECKIN_ALREADY_CONFIRMED_MESSAGE)
- Câmera bloqueada / permissão necessária
- Evento indisponível (vários motivos + hints SQL)
- Aponte para o QR Code da família...

3.12 Manutenção (/maintenance-dashboard)

Serve para: operação interna — eventos, salas, quórum, escalas, pastoral, financeiro, ACL, cadastro de usuários.

Menu de módulos (atalhos)

Módulo	Componente	Quem acessa
Programação de Eventos	Lista + editor	Staff com acesso manutenção
Cronograma de Eventos	Gantt	Idem
Sala(s) - Check In	MaintenanceSalaMonitorCard	Staff — marca entrada Kids/Teens
Lista de Presença	MaintenanceQuorumPresenceCard	Staff — leitura pós-totem
Tipos de Escala	MaintenanceScaleTypesCard — vagas/domingo e modo ciclo	ACL escala
Servos em Disponibilidade	MaintenanceScaleVolunteersCard	ACL escala
Programação de Escalas	MaintenanceScalesCard	Ciclo em bloco via aplicar_ciclo_escala
Cuidado Pastoral	MaintenancePastoralCareCard	Papel pastoral
Informações Financeiras	MaintenanceFinancialsCard	Import CSV, esvaziar mês REALIZADO
Controle de Acesso	MaintenanceAccessControlCard	super_admin
Cadastro de Usuário	MaintenanceProfileCadastroCard	super_admin

Editor de eventos (campos)

Campo	Função
Nome, data/hora, local	Identificação
Capacidade	Vagas obrigatórias

Chips Kids / Teens / Ofertas	Recursos do evento
Ativação de Totem	Habilita fluxo totem
Requer Quorum	Fluxo quórum + lista de presença
Publicação	Publicado / Rascunho
Tabela quórum	Registro em tempo real (poll 15 s)
Salvar / Apagar / Cancelar	CRUD

Toasts: evento criado/atualizado/apagado; erros de formulário/RLS.

4. Fluxos de negócio — quem inicia e quem termina

Processo	Inicia	Termina	Onde
Login membro	Usuário	App grava sessão	/
Primeiro PIN	Usuário (WhatsApp)	RPC grava PIN temporário	/
Cadastro inicial	Sistema redireciona	Perfil + family_id criados	/register
Completar perfil	Onboarding	Membro salva campos	/manage-profile
Audiência / pré-check-in	Membro (checkbox)	Membro ou totem (quórum trava)	Dashboard Agenda
Check-in totem	Membro mostra QR	Totem confirma via RPC	/totem-checkin
Check-in sala Kids/Teens	Membro inscreve na audiência	Staff marca checkbox	Manutenção Salas
Quórum lista presença	Totem confirma	Staff imprime/consulta	Manutenção Lista Presença
Pedido pastoral	Membro envia	Equipe pastoral (fora do app)	/pastoral
Financeiro leitura	Membro consulta	—	/financial
Financeiro carga	Staff importa CSV	RPC manutenção	Manutenção Financeiro
Escala ciclo	Staff preview + confirma	RPC aplicar_ciclo_escala	Manutenção Escalas
ACL papéis	Super admin	Grants no Supabase	Manutenção ACL
Mapa geolocalização	Membro/staff abre mapa	Sync CEP → pins	PWA mapa
Logout	Usuário (Sair)	Sessão limpa	Dashboard / totem

Diagrama — check-in completo (totem + salas)

```
sequenceDiagram
    participant Staff as Staff (Manutenção)
    participant Membro as Membro (Dashboard)
    participant Totem as Totem (Quiosque)
    participant DB as Supabase

    Staff->>DB: Cria evento (totem/quórum/salas)
    Membro->>DB: Marca audiência (pré-check-in)
    Membro->>Membro: Exibe QR no card Check-in
    Totem->>DB: lookup_totem_checkin
    Totem->>DB: confirm_totem_checkin
    DB-->>Membro: Audiência travada (quórum)
    Staff->>DB: Marca entrada sala Kids/Teens
    Staff->>DB: Consulta Lista de Presença
```

5. Catálogo consolidado de mensagens interativas

Alerts (bloqueantes)

Login, cadastro, ACL negado, validações de formulário, confirmações destrutivas (excluir membro/evento), erros de RPC ausente, PIX, WhatsApp, câmera, PIN, totem, pastoral, perfil, família, veículos.

Toasts (totem + manutenção)

Confirmação check-in, já confirmado, sucesso/erro de salvamento em manutenção.

Banners / hints inline (não bloqueantes)

Gate pré-check-in, quórum, LGPD pendente (header vermelho), ACL indisponível, meses só planejado, comentários financeiros, schema SQL pendente na manutenção, cache do mapa, estados vazios de listas.

Confirmações (dupla ação)

Apagar evento (Sim, apagar / Não), excluir membro, substituir selfie.

6. Scripts SQL e dependências de deploy

Ordem recomendada no Supabase:

1. `scripts/access-control-role-display-order.sql`

2. `scripts/access-control-schema.sql` + seeds ACL
3. `scripts/escalas-multi-vagas.sql`
4. `scripts/escalas-integrity-constraints.sql`
5. `scripts/escalas-apply-cycle-batch.sql` (inclui `aplicar_ciclo_escala` + `get_scale_cycle_context`)
6. `scripts/escalas-tipos-maintenance-rpc.sql`
7. `scripts/escalas-volunteers-rpc.sql`
8. `scripts/escalas-maintenance-rpc.sql`
9. `scripts/checkins-totem-flow.sql` + `scripts/events-quorum-registry.sql`
10. `scripts/profiles-sync-address-from-cep-rpc.sql`
11. `scripts/access-control-map-screen.sql`
12. `scripts/financials-maintenance-rpc.sql`
13. `scripts/verificar-login.sql` , scripts PIN/LGPD conforme ambiente

Produção:

- `EXPO_PUBLIC_ACL_STRICT=true`
- Deploy PWA: `npm run build:web` → publicar `dist/` em HTTPS

7. Resumo de confiança e proteção

A solução combina:

- **Autenticação por PIN** validado no servidor (`verificar_login`)
- **Autorização granular** (tela / card / coluna / tabela) via `profile_has_access`
- **RLS no PostgreSQL** com identidade transportada por header `x-profile-id`
- **RPCs `security_definer`** para operações sensíveis (PIN, check-in, escala em lote, exclusão financeira escopada)
- **Isolamento do totem** (aparelho dedicado, sem ACL de tela)
- **LGPD** com registro auditável (`lgpd_accepted`)
- **Separação leitura (membro) vs escrita (manutenção)** em finanças e eventos

Dados residem no Supabase (PostgreSQL gerenciado). O app cliente **nunca** recebe chave `service_role` .

8. Inventário de cards do dashboard (ACL)

resource_key	Card	Rota / destino
<code>screen:dashboard.card.event_alt</code>	Agenda da Família	Inline no dashboard

screen:dashboard.card.qr	Check In / QR Totem / Quórum	Inline + modal
screen:dashboard.card.kids_teens	SALA(S)	Inline (leitura)
screen:dashboard.card.offerings	Dízimos e Ofertas	Inline
screen:dashboard.card.pastoral	Coração Aberto	/pastoral
screen:dashboard.card.members_list	Lista de Membros	Inline + /mapa-geolocalizacao
screen:dashboard.card.birthdays	Aniversariantes	Inline
screen:dashboard.card.financial	Financeiro	/financial
screen:dashboard.card.vigilance_scales	Escalas	Inline (roster)
screen:dashboard.card.parking_vehicle_v2	Estacionamento	Inline
screen:dashboard.card.grouped_manage	Dados + Família	/manage-profile, /manage-members

Documento gerado a partir do código-fonte e da documentação do projeto app-igreja.